ПМ.13 · 1. УП.ПМ.13 · оформление и проверка

Полное общее задание

1

Назначение и результат

Оформить один игровой проект назначенного варианта: связать визуальные роли, подписи, интерфейс, обратную связь, меню, паузу и результат с работающим правилом. Сохранить номер, область, палитру, сущности и обязательный сигнал из отдельного списка предметных областей. Разбор «Кристального маршрута» показывает приём и не является второй обязательной работой.

2

Среда и исходные знания

Работать в Unity 6.6 (6000.6.2f1), C# и Unity UI (Legacy Text). Нужны доступный Unity Editor, редактор C# и Word для отчёта. Исходный архив содержит готовую механику и сохранённую сцену. Освоить выбор объекта в Hierarchy, правку полей Inspector и сохранение файла; расположение и действия раскрыты далее. Не требуется создавать новую игру или получать результат другой практики.

3

Исходные условия

Получить CrystalRoute-starter.zip из раздела «Файлы для работы», распаковать и открыть проект. Сохранить Main как Student; вне Play задать Variant Number от 1 до 30. Все профили находятся в одном исходном проекте, отдельный архив варианта не требуется. Доработать представление исходной механики своего профиля, сохранив его правило.

4

Образец и индивидуальная работа

Использовать область «Кристальный маршрут» с номером 0 для чтения разбора. Оформленный проект CrystalRoute-demonstration.zip открывать в отдельной папке после собственного изменения для сравнения. Числа, имена и сигналы образца заменить условиями своего варианта.

ПМ.13 · 2. УП.ПМ.13 · оформление и проверка

Наблюдаемый результат

1

Сцена

Игрок, предметы, предупреждения и зона выхода различаются цветом, формой и подписью. После остановки Play объекты остаются в Hierarchy, а сохранённые изменения сохраняются при новом запуске.

2

Интерфейс

HUD показывает название, прогресс и жизни; для переноса — груз, для последовательности — следующий номер, для комплекта и переключателей — отдельные отметки. Подсказка сообщает причину неправильного действия.

3

Обратная связь

Успешное действие, предупреждение, победа и поражение имеют различимые сигналы. Вращение и пульсация не изменяют коллайдер. Звук запускается событием, а не каждый кадр.

4

Экраны

Работают кнопки «Играть», «Пауза», «Продолжить», «В меню» и «Заново». Пауза останавливает движение; повторный запуск восстанавливает прогресс, груз, жизни и доступные предметы.

5

Индивидуальное оформление

Выполнить особый сигнал карточки: анимацию, грузовые ячейки, независимые отметки, шкалу ресурса или индикаторы доступа по своему условию. Сохранить игрока, предмет, опасность и кнопку как префабы. Внешние ресурсы сопровождать источником и условиями использования; разрешены собственные простые изображения и синтезированные сигналы основы.

ПМ.13 · 3. УП.ПМ.13 · оформление и проверка

Последовательность работы

1

Исходные условия

Открыть Main, выбрать свой номер Variant Number на GameManager, запустить проект и проверить правило из карточки варианта. Зафиксировать, что уже работает, и какие элементы оформления не помогают понять состояние.

2

Причина и изменение

Найти ответственный элемент: VisualTheme задаёт палитру, ThemeRole связывает цвет с ролью, HUDView отображает данные, GameManager сообщает события, ScreenMockupController переключает экраны. Менять оформление в нужном месте.

3

Повторная проверка

После каждого законченного изменения повторить затронутый сценарий. Затем проверить прежние сценарии: правильное и неправильное действие, выход до достижения цели, паузу, победу, поражение и повторный запуск.

4

Передача пользователю

В меню сформулировать цель простыми словами, в подсказках — причину отказа. При сдаче проекта приложить ссылку на его репозиторий. Фактические результаты и ограничения описать в общем шаблоне отчёта.

ПМ.13 · 4. УП.ПМ.13 · оформление и проверка

Проверочные сценарии

1

Проверка начального состояния

До нажатия «Играть» видно меню; после нажатия — HUD и поле. Игрок перемещается стрелками/WASD и экранными кнопками. При отпускании экранной кнопки движение прекращается.

2

Проверка правила

Выполнить принятую и отклонённую операцию из своего варианта. Сверить изменение счётчика, груза, отметок и текста причины. Вход в выход без выполнения цели не должен давать победу.

3

Проверка экранов

Пауза не меняет прогресс и жизни, продолжение восстанавливает управление. Победа и поражение имеют разные заголовки. «Заново» сбрасывает состояние.

4

Проверка масштабирования

В Game View проверить 1280×720, 1920×1080 и 720×1280. Текст читается, переносится и остаётся внутри панелей; элементы не накладываются. При необходимости увеличьте высоту текста и поправьте якоря.

5

Проверка сохранения

Остановить Play, сохранить сцену Cmd+S/Ctrl+S, повторно открыть её и запустить. Изменения оформления должны остаться, в Console не должно быть ошибок проекта.

6

Полная попытка и сброс

Подготовить поражение: трижды коснуться Hazard с интервалом больше одной секунды; после каждого контакта выйти из его области и войти снова. Для вариантов 17, 18, 20 отдельно дождаться исчерпания ресурса. Проверить победу, поражение, «Заново» и «В меню»: прогресс 0, груз 0, жизни 3, все предметы доступны, отметки сняты, ресурс 90 при его наличии.

ПМ.13 · 5. УП.ПМ.13 · оформление и проверка

Содержательные критерии результата

1

Целостность

Цвет, форма, типографика и отступы согласованы на сцене и экранах. Назначение объектов различимо без опоры только на цвет.

2

Понятность

По меню, HUD и обратной связи можно объяснить цель и текущее состояние именно данного варианта.

3

Работоспособность

Ссылки компонентов и события кнопок настроены; правильное действие, отказ, пауза, победа, поражение и повторный запуск работают после сохранения сцены.

4

Обоснование

Объяснить связь выбранного оформления с задачей. Для внешних изображений и звуков указать прямой источник и условия использования. Собственные материалы обозначьте как созданные автором работы.

Состав и порядок сдачи

1

Проект и материалы

Представить репозиторий собственного проекта с Assets, Packages, ProjectSettings и README: версия Unity, путь Student, номер варианта, управление и порядок открытия. Сохранить .meta рядом с ресурсами. Исключить Library, Temp, Logs, obj, UserSettings и Build. Включить используемые изображения, звуки, префабы, сцену и описание источников.

2

Отчёт и документы

Заполнить отдельный «Отчет о выполнении.docx» из пакета карточки: исходные условия, собственные изменения и доказательства, ожидаемые и фактические результаты, итоговая проверка и ссылка на репозиторий. Заполнить предоставленные индивидуальное задание и аттестационный лист; подготовить презентацию по форме пакета. Остальные установленные формы карточки сохраняются в составе выдачи.

3

Доказательства

Сохранить сцену вне Play; показать начальное состояние и выбор профиля, палитру и префабы, принятую и отклонённую операцию, особый сигнал, меню, паузу, победу, поражение, три разрешения и повторное открытие. Для анимации дополнить кадр описанием изменения во времени. В форме предусмотрены места для этих иллюстраций.

4

Сборка и передача

При установленном модуле платформы собрать Student и проверить отдельное приложение; при отсутствии модуля указать ограничение в отчёте. Передать заполненные документы и доступную проверяющему ссылку. Открыть ссылку в отдельном браузерном сеансе и убедиться, что видны файлы проекта.

ПМ.13 · 7. Оформление игрового продукта

Файлы для работы

↓

Скачивания

Скачать исходный Unity-проект с готовыми правилами 30 вариантов (ZIP)

Скачать оформленный «Кристальный маршрут» для сверки (ZIP)

Скачать форму отчёта (DOCX)

1

Исходный проект

CrystalRoute-starter.zip — рабочая основа с готовыми правилами всех 30 вариантов. Распаковать, открыть starter-project, сохранить Student. Начать с этого архива; новый проект с нуля не требуется.

2

Оформленный тестовый проект

CrystalRoute-demonstration.zip — отдельный пример «Кристальный маршрут». Распаковать в другую папку, открыть unity-project/Assets/Scenes/Demonstration.unity. Использовать для сверки после собственного изменения; Main в этом архиве также оформлена. Не смешивать Assets двух проектов.

3

Форма отчёта и документы

Отчет о выполнении.docx — чистая редактируемая форма с местами для доказательств. В пакете карточки EatDoners также выдаются установленные формы и презентация. Предметные области — отдельный PDF; выбрать только свой номер.

ПМ.13 · 8. УП.ПМ.13 · оформление и проверка

Как устроена работа

1

Общая задача

Образец «Кристальный маршрут» использует пять правильных предметов, один предупреждающий, опасности и выход. Рабочий проект начать с нейтрального исходного архива; показанные далее цветные кадры относятся к отдельному оформленному примеру. Сначала проверить состояние, затем изменить один элемент оформления и повторить проверку. Работать сразу со своим номером, применяя приёмы образца.

2

Одна рабочая сцена

Сцена — сохранённый набор объектов. Hierarchy перечисляет объекты открытой сцены; Inspector показывает компоненты выделенного объекта; Project содержит файлы. Открыть окна через Window → General → Hierarchy / Inspector / Project, если они скрыты. В этой основе GameBootstrap не создаёт объекты при запуске; GameManager использует назначенные ссылки. Поэтому редактировать нужно сохранённую Student вне Play.

3

Самостоятельное оформление

Для назначенного варианта самостоятельно выбрать композицию и детали оформления в рамках его палитры и обязательного сигнала. Из карточки выписать номер, сущности, правило, цель, вместимость и ресурс. Номер 0 и кристаллы — только пример; готовый оформленный архив не становится индивидуальным решением.

4

Как читать справочник

В конце методички приведён исходный код основы по файлам и методам. Это справочник для поиска существующих частей, а не требование переписать каждый файл. Учебные изменения показаны отдельно в маршруте; в собственном проекте выполнять только нужные замены.

ПМ.13 · 9. УП.ПМ.13 · оформление и проверка

Загрузка и первое открытие

1

Распаковать исходный проект

Скачать CrystalRoute-starter.zip по отдельной ссылке раздела файлов. Распаковать, затем открыть внутреннюю папку starter-project: в ней одновременно лежат Assets, Packages и ProjectSettings. В Unity Hub выбрать Projects → Add → Add project from disk и указать именно starter-project, а не внешнюю папку распаковки и не Assets. При запросе версии выбрать указанную в задании; дождаться импорта.

2

Открыть Main

В Unity Project раскрыть Assets → Scenes, дважды щёлкнуть Main. До Play в Hierarchy видны Main Camera, GameManager, Player, Item-1...Item-6, Hazard-1...Hazard-3, Exit, Canvas и EventSystem.

3

Проверить исходное правило

Проверить номер 0 до собственного выбора: нажать Play в верхней панели, затем «Играть». Подвести Player стрелками/WASD к №1: прогресс 1/5. Коснуться №6 с «!»: прогресс остаётся 1/5, появляется предупреждение. Войти в Exit раньше 5/5: победы нет. Остановить Play той же кнопкой: объекты остаются в Hierarchy. После этого задать свой номер и проверить сценарий его карточки.

4

Сохранить рабочую сцену

Остановить Play. File → Save As: выбрать Assets/Scenes, имя Student.unity. В Project дважды открыть Student и проверить её имя в Hierarchy. Нажать Ctrl+S (Windows) или Cmd+S (macOS). Дальше менять эту сцену вне Play. Настройка объекта в Play временная: записать нужное значение, остановить запуск, внести его снова и сохранить.

5

Оформленный пример: начало попытки



Рисунок 1 — Оформленный «Кристальный маршрут»: прогресс 0/5, три жизни, предметы и выход. В исходном Main палитра нейтральная; совпадения цветов до оформления не требуется.

Связи данных и интерфейса

1

Кто хранит данные

Выбрать GameManager в Hierarchy. В одноимённом компоненте назначены Player, Hud, Screens, Theme, Audio Feedback, Items, Profiles и Gate Indicators. Items содержит шесть объектов, Profiles — Assets/profiles.json.

2

Кто показывает данные

Canvas содержит HUDView и ScreenMockupController. GameManager вызывает ShowProgress после изменения состояния и ShowResult при завершении. Здесь нет второго HUD через OnGUI: видимый интерфейс и данные связаны.

3

Где посмотреть причину

Дважды щёлкнуть Assets/Scripts/GameManager.cs. Найти Collect → Refresh → HUDView.ShowProgress; TryFinishLevel → Finish → HUDView.ShowResult → Screens.ShowResult. Эти связи уже включены в исходник. Не дописывать повторные вызовы звука.

4

Проверка связей

В Play собрать один предмет. ProgressText меняется один раз. В Console нет NullReferenceException. Если поле пустое, остановить Play и назначить сохранённый объект сцены, а не объект другого открытого проекта.

ПМ.13 · 11. УП.ПМ.13 · оформление и проверка

Палитра и смысловые роли

1

Выбрать набор цветов

Палитра — ограниченный набор цветов с постоянным смыслом. В демонстрации фон тёмный, игрок голубой, цель жёлтая, опасность красная, выход зелёный. Записать цвета и объяснить назначение каждого.

2

Задать палитру

Остановить Play. Выбрать GameManager → VisualTheme. Открыть цветное поле; в строке HEX ввести значение и закрыть окно. Для образца: Background #09101A, Surface #121A2E, Player #30E0FF, Target #FFC729, Danger #FF4047, Success #40D172, Text #FFFFFF, Accent #30E0FF. Для своего варианта подобрать оттенки заданной палитры и записать соответствие роли и цвета. Сохранить Student.

3

Назначить роль объекту

Player имеет ThemeRole → Player. Item-N → Visual имеет Target, Hazard-N — Danger, Exit — Success. Если заменяете изображение, сохранить этот компонент. Theme.Apply применяет цвет при старте ко всем назначенным ролям, включая скрытые панели.

4

Почему правка цвета может исчезнуть

Не менять только Sprite Renderer.Color: при новом запуске VisualTheme снова применит роль. Для постоянного изменения правьте поле палитры или ThemeRole. Сериализованное значение Inspector имеет приоритет над начальным значением поля в C#.

5

Проверка

Запустить, сравнить роли, остановить и запустить снова. Цвета повторяются. В оттенках серого роли должны отличаться силуэтом, значком или подписью.

ПМ.13 · 12. УП.ПМ.13 · оформление и проверка

Файлы оформления и постепенная правка кода

1

Открывайте существующий файл

В Project → Assets → Scripts дважды щёлкнуть VisualTheme.cs. Эти классы уже включены в исходный проект: не создавать второй файл с тем же классом. Автоматический шаблон MonoBehaviour здесь не нужен.

2

Порядок изменения

Найти метод или поле по имени из инструкции. Заменять только указанную логическую часть, сохранять Cmd+S/Ctrl+S, возвращайтесь в Unity и дождаться компиляции. Полные версии ниже взяты из оформленного примера и служат для сверки границ класса, а не требуют копирования всех файлов одновременно.

3

Проверка законченного изменения

После изменения одного метода проверить его результат. Красная строка Console означает, что дальше продолжать нельзя: двойной щелчок открывает указанную строку. Сначала исправьте первую ошибку — последующие могут быть её следствием.

ПМ.13 · 13. УП.ПМ.13 · оформление и проверка

Смена изображений без повреждения механики

1

Импорт собственного PNG

В Assets создать папку Art/MyVariant через правый щелчок → Create → Folder. Перетащить PNG в эту папку. Выбрать его: Texture Type → Sprite (2D and UI), Sprite Mode → Single, Pixels Per Unit → 64. Для пиксельной графики Filter Mode → Point, для гладкой → Bilinear; нажать Apply.

2

Менять видимый дочерний объект

Раскрыть Item-1 → Visual. Перетащить новый спрайт в Sprite Renderer → Sprite. Масштаб изображения менять на Visual. На родительском Item-1 оставить Collectible и Collider: правило и столкновения не должны вращаться вместе с рисунком.

3

Игрок и опасность

Выбрать Player: заменить Sprite, сохранить Rigidbody 2D и PlayerController. Body Type — Dynamic, Gravity Scale — 0, Freeze Rotation Z включён. На Hazard оставить триггер. Edit Collider позволяет подогнать границу к видимому объекту.

4

Источник подписей предметов

GameManager.SelectVariant при каждом старте заново задаёт Number.Text и роли Target/Danger. Чтобы сохранить названия, открыть GameManager.cs и найти присваивание items[i].transform.Find("Number")...text внутри SelectVariant. Для сохранения номеров оставить эту строку; тематические названия указать в profile.goal (меню) и отдельной подписи. Для замены самих номеров использовать способ в разделе «Подписи, которые сохраняются после запуска». Простая правка Number в Inspector будет перезаписана.

5

Проверка

Проверить касание предмета, опасности и выхода. После изменения Scale игрок не должен проходить через стену или останавливаться далеко от неё. Оформление не меняет целевое число и правило приёма.

ПМ.13 · 14. Оформление игрового продукта

Подписи, которые сохраняются после запуска

1

Изменить существующий источник

В GameManager.cs внутри SelectVariant найти цикл for по items. Сразу после штатного присваивания Number.Text, до закрывающей скобки этого цикла, добавить одну строку ниже. Это пример для варианта 6; для своего номера заменить условие и шесть подписей. Индексы массива идут 0–5. Для комплектов и доступа повторить название типа согласно i % profile.target. В остальных местах не создавать второй SelectVariant.

2

Пример замены подписей

```
if (number == 6)
    items[i].transform.Find("Number")
        .GetComponent<TextMesh>().text =
        new[] { "I", "II", "III", "IV", "V", "Лишняя" }[i];
```

3

Сохранить и проверить

Сохранить C#; в Unity дождаться компиляции. Вне Play оставить Variant Number = 6, сохранить Student. Запустить и сравнить I–V с порядком приёма; после остановки и нового запуска подписи должны повториться. Для своего номера сохранить смысл предупреждающего объекта или повторного типа. Если все предметы получили одно имя, проверить индекс [i].

ПМ.13 · 15. УП.ПМ.13 · оформление и проверка

Префабы и сохранение изменений

1

Что такое префаб

Префаб хранит повторно используемый объект вместе с компонентами. Изменение образца можно перенести на экземпляры. В исходном проекте уже есть Player, Item, Hazard и Button в Assets/Prefabs.

2

Сохранить свой образец

Остановить Play. Создать Assets/Prefabs/MyVariant. Перетащить Item-1 из Hierarchy в эту папку и назвать VariantItem. В образец попадут Visual, Number, Collider, Collectible и CollectibleVisual.

3

Сохранить идентификаторы

Если заменяете остальные предметы экземплярами своего префаба, задать каждому Collectible.Index: 0,1,2,3,4,5. Добавить новые экземпляры в соответствующие позиции GameManager.Items. Нельзя оставлять одинаковый Index у всех: последовательность и типы станут неверными.

4

Сохранить общие и индивидуальные значения

Общие Sprite и Scale править в Prefab Mode: двойной щелчок по VariantItem в Project, выбрать его Visual, внести изменение, сохранить и выйти стрелкой назад в Scene. На экземплярах Student оставить индивидуальные Collectible.Index. Если используется меню Overrides экземпляра, раскрыть список и применить только нужное свойство Visual; не выбирать Apply All для экземпляра с собственным Index.

5

Проверка

Сохранить Student, повторно открыть её и проверить все шесть ссылок Items. Префаб — средство повторного оформления; ссылки в GameManager должны указывать на экземпляры сцены.

ПМ.13 · 16. УП.ПМ.13 · оформление и проверка

Анимация видимой части предмета

1

Найти компонент

Выбрать Item-1 → CollectibleVisual. Поле Visual указывает на дочерний Transform, Rotation Speed — скорость вращения, Pulse Amount — доля изменения масштаба, Pulse Speed — частота пульсации.

2

Добавление при необходимости

Если компонент удалён, нажать Add Component → CollectibleVisual на родительском Item-1. Перетащить дочерний Visual в поле Visual. Задать 70, 0.08 и 4. Сохранить сцену. Для неподвижной цели задать Rotation Speed = 0.

3

Логическая часть кода

В Assets/Scripts/CollectibleVisual.cs сначала разобрать OnEnable: он сохраняет initialScale. Затем Update: Rotate использует deltaTime, Sin изменяет масштаб вокруг начального. OnDisable восстанавливает масштаб и поворот; без этого повторный запуск может накопить искажение.

4

Проверка

Запустить: изображение вращается, родительский Collider остаётся неподвижным. Собрать предмет, перезапустить — прежний масштаб восстановлен. Не назначать корневой Item в поле Visual.

ПМ.13 · 17. УП.ПМ.13 · оформление и проверка

Canvas и масштабирование

1

Проверить существующий Canvas

Не создавать второй Canvas. Выбрать Canvas: Render Mode → Screen Space - Overlay. Canvas Scaler: Scale With Screen Size, Reference Resolution 1280×720, Match Width Or Height, Match = 0. EventSystem в сцене один.

2

Якоря

Якоря — точки привязки Rect Transform к родителю. Canvas/GamePanel/HUDPanel растянут по верхнему краю: Anchor Min (0,1), Max (1,1), Left 16, Right 16, Top 10, Height 132. Это сохраняет ширину панели при смене размера.

3

Почему выбран Match = 0

Масштаб привязан к ширине, поэтому длинные подписи не выходят за боковые края портретного экрана. CameraFit отдельно сохраняет всё поле в камере; уменьшение поля в портретном виде не изменяет правило.

4

Проверка

Game View → список разрешений → плюс: создать Fixed Resolution 1280×720 и 720×1280. Проверить оба. Если текст обрезан, изменить высоту Rect Transform и размер шрифта, а не отключать масштабирование.

ПМ.13 · 18. УП.ПМ.13 · оформление и проверка

HUD: точные поля и привязки

1

Иерархия

Canvas → GamePanel → HUDPanel содержит TitleText, ProgressText, LivesText. HintText расположен непосредственно в GamePanel. Панель результата заранее существует рядом с GamePanel и содержит ResultTitle, ResultMessage, RestartButton.

2

Компонент текста

Исходный проект использует UnityEngine.UI.Text — UI → Legacy → Text. Не подставлять TextMeshPro в поля типа Text. Для нового текста выбрать Canvas/GamePanel → правый щелчок → UI → Legacy → Text; назвать HintText и настроить перенос по ширине.

3

Ссылки HUDView

Выбрать Canvas → HUDView: Title Text ← HUDPanel/TitleText, Progress Text ← ProgressText, Lives Text ← LivesText, Hint Text ← GamePanel/HintText, Result Title/Message ← ResultPanel/ResultTitle и ResultMessage. Menu Title/Goal ← MainMenuPanel/MenuTitle и MenuGoal.

4

Размеры

TitleText: Font Size 28, Alignment Middle Center, верхняя часть панели. ProgressText: 23, нижняя часть, перенос включён. LivesText: 24 справа сверху. HintText: 22, внизу справа от управления. При длинном названии уменьшать размер или увеличивать область, сохраняя читаемость.

5

Проверка

Собрать предмет и коснуться опасности. Изменяются соответствующие поля. При пустом значении искать неназначенную ссылку; при неизменном показателе — GameManager.Hud и вызов Refresh, а не цвет текста.

ПМ.13 · 19. УП.ПМ.13 · оформление и проверка

Видимый сигнал правильного и неправильного действия

1

Причина различения

Изменение счётчика подтверждает принятую операцию; предупреждение объясняет отказ. Цвет дополняется текстом и символом, чтобы сигнал оставался понятным без цветового различения.

2

Место правки

Открыть Assets/Scripts/HUDView.cs, найти ShowHint. Заменить его тело двумя строками из соответствующего листинга: сначала присвоить текст с «!» или «✓», затем выбрать цвет по warning. Сигнатуру ShowHint(string message, bool warning) сохранить: её использует GameManager.

3

Зачем нужен параметр

warning передаёт результат проверки правила, а не угадывается по содержимому строки. Collect вызывает ShowHint с false, Warn — с true. Это позволяет менять текст и язык, не ломая выбор сигнала.

4

Проверка

До победы коснуться правильного предмета и №6. У принятого действия прогресс растёт; у отклонённого — остаётся прежним, причина выделяется символом. Проверить, что сигнал не закрывает управление.

5

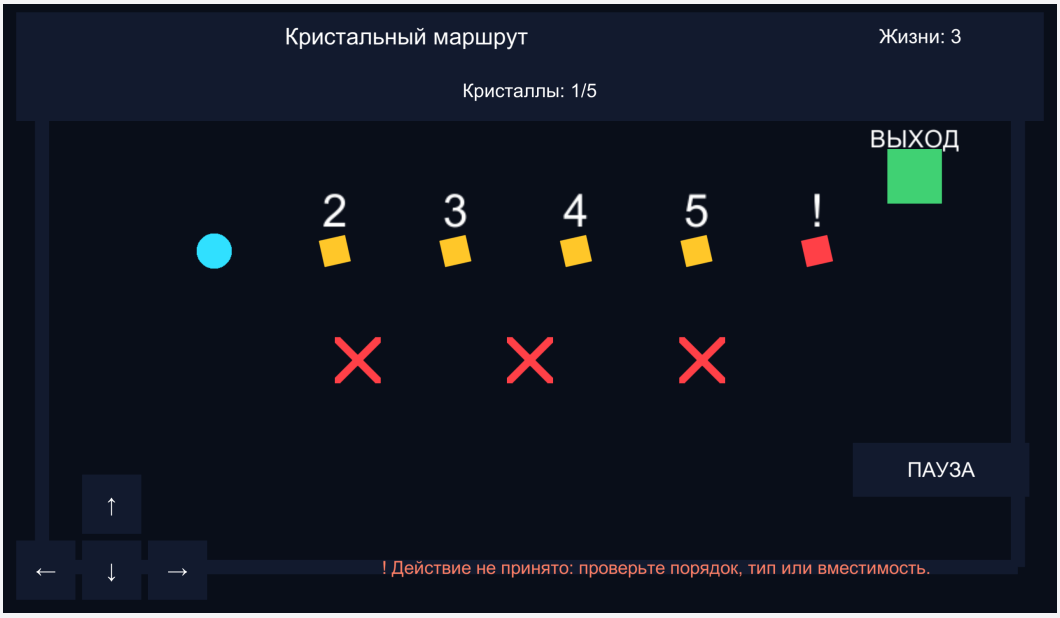
Принятый предмет изменил прогресс.



Принятый предмет изменил прогресс.

6

Отклонённая операция сохранила прогресс и показала причину.



Отклонённая операция сохранила прогресс и показала причину.

Точная причина отказа и завершения

1

Найти общий отказ

В GameManager.Collect уже вычислено bool wrong. Его выражение не менять. Заменить только строку if (wrong) { Warn(...); return; } приведённым блоком: четыре причины выбираются по тому же mode. Правильный подбор после блока остаётся прежним.

2

Причина для своего правила

```
if (wrong)
{
    string reason;
    if (profile.mode == 0)
        reason = "Этот тип не нужен для цели.";
    else if (profile.mode == 1)
        reason = "Сначала нужен предмет №" + (progress + 1);
    else if (profile.mode == 2 || profile.mode == 3)
        reason = "Груз заполнен: сначала выполнить доставку.";
    else
        reason = "Этот тип уже отмечен: выбрать другой.";
    Warn(reason);
    return;
}
```

3

Назвать причину поражения

В HUDView.ShowResult заменить присваивание resultMessage.text строками ниже. После победы выводится цель, при нуле жизней — причина урона, иначе — исчерпание ресурса. Тексты заменить терминами своей области, не менять условие выбора.

4

Сообщение результата

```
resultMessage.text = won ? "Цель выполнена."
    : GameManager.Instance.Lives == 0
        ? "Жизни закончились: избегать опасностей."
        : "Ресурс исчерпан: сократить маршрут.";
```

5

Проверить различение

Запустить принятую и отклонённую операции по матрице своего mode. Сравнить прогресс до и после отказа — он прежний, причина соответствует подготовленному случаю. Затем отдельно получить поражение от опасностей и, где есть ресурс, от ожидания. Снимок прежнего примера с общей строкой отказа иллюстрирует исходное состояние; после изменения в отчёт поместить собственный точный текст.

ПМ.13 · 21. Оформление игрового продукта

Событие, анимация и возврат к исходному виду

1

Подготовить видимый индикатор

Остановить Play. Для краткой галочки, рамки или вспышки на Canvas/GamePanel создать UI → Image, назвать FeedbackVisual, задать соответствующий спрайт и положение вне кнопок. У Image снять Raycast Target. Для эффекта предмета выбрать его дочерний Visual; не анимировать корень с Collider. Добавить дочернюю рамку или знак при необходимости. Постоянно видимые элементы не деактивировать.

2

Записать клипы

Открыть Window → Animation → Animation, выделить нужный визуальный объект. Create → сохранить Assets/Animations/MyVariant/Idle.anim. В записи (красная кнопка Record) задать исходный цвет, Scale и положение на времени 0, выключить Record. Через список клипов → Create New Clip создать Signal.anim. Samples = 30. Для вспышки задать Alpha 0,1,0 на кадрах 0,4,12; для толчка Scale 1,1.12,1; для встряхивания Local Position X 0,+8,-8,0 на кадрах 0,3,6,12. Значения смещения UI — пиксели, у спрайта сцены использовать ±0.08 единицы. В Project у Signal снять Loop Time. Сохранять исходные значения в последнем кадре.

3

Связать состояния

Window → Animation → Animator. Idle назначить состоянием по умолчанию (Set as Layer Default State). В Parameters добавить Trigger Signal. Создать переход Idle → Signal: Has Exit Time выключен, Condition Signal; переход Signal → Idle: Has Exit Time включён, Exit Time 1, Transition Duration 0. Animator.Update Mode = Unscaled Time, если эффект должен закончиться на экране результата при Time.timeScale = 0.

4

Привязать вызов к событию

В GameManager добавить поле public Animator acceptedVisual, warningVisual; вне Play назначить Animator соответствующих объектов. В успешной ветви Collect перед item.gameObject.SetActive(false) вызвать acceptedVisual.SetTrigger("Signal"); в Warn до hud.ShowHint — warningVisual.SetTrigger("Signal"); при необходимости проверять ссылку через if (acceptedVisual != null). Использовать один и тот же внешний индикатор для разных предметов; для исчезновения самого предмета применять следующий раздел. Не добавлять звук здесь: он уже вызывается событием.

5

Настроить варианты клипа

Свечение — полупрозрачная рамка за спрайтом с Alpha 0.2→0.8→0.2. Затемнение/красная рамка — UI Image поверх поля с Raycast Target выключен и Alpha 0→0.25→0. Расходящиеся кольца — два дочерних кольца: Scale 1→1.8, Alpha 1→0, второе начало на 3 кадра позже. Последовательное гашение — ключи Alpha у пяти дочерних значков со сдвигом 2 кадра, к концу вернуть все к исходному виду. Штриховка/пунктир — подготовить собственный PNG из коротких полос и анимировать его Alpha; заменять только оформление индикатора, физические стены оставить.

6

Проверить цикл

Запустить событие дважды с паузой больше длины клипа: каждый раз эффект начинается и заканчивается одинаково. После «Заново» нет растущего масштаба и залипшего затемнения. На паузе управление не двигает игрока. Если эффект не начинается, сверить имя Signal, условие перехода и ссылку Animator; если остаётся включённым — последний кадр и переход в Idle.

ПМ.13 · 22. Оформление игрового продукта

Исчезновение предмета после принятия

1

Сохранить правило и видимый эффект

GameManager сразу скрывает принятый предмет. Поэтому анимация на нём оборвётся. Для растворения варианта 5 создать копию только его Visual (без Collider/Collectible) в Assets/Prefabs/MyVariant/FadeVisual; удалить CollectibleVisual, чтобы два компонента не меняли Scale одновременно. Создать Signal с SpriteRenderer.Color Alpha 1→0 за 0.4 секунды, Loop Time выключен. Animator.Update Mode = Unscaled Time.

2

Добавить законченный компонент

Создать Assets/Scripts/FadeVisual.cs через Create → Scripting → MonoBehaviour Script; заменить весь автоматически созданный текст листингом. Компонент удаляет только временную картинку после окончания сигнала, а не объект игрового правила.

3

FadeVisual.cs

```
using UnityEngine;
public class FadeVisual : MonoBehaviour
{
    private float remaining = 0.5f;
    private void OnEnable()
    {
        GetComponent<Animator>().SetTrigger("Signal");
    }
    private void Update()
    {
        remaining -= Time.unscaledDeltaTime;
        if (remaining <= 0f) Destroy(gameObject);
    }
}
```

4

Назначить и вызвать

На FadeVisual добавить этот компонент. В GameManager объявить public GameObject acceptedFade; и public Transform acceptedPoint;. Назначить префаб и объект Exit как Accepted Point. В успешной ветви Collect непосредственно перед item.gameObject.SetActive(false) добавить строку ниже. Для варианта 5 копия появляется в узле передачи и растворяется; исходное скрытие и прогресс сохранить.

5

Вызов в Collect

```
if (acceptedFade != null && acceptedPoint != null)
    Instantiate(acceptedFade, acceptedPoint.position,
        Quaternion.identity);
```

6

Проверить

Принять предмет: прогресс меняется сразу один раз, временная картинка растворяется и исчезает, повторный контакт не даёт второго зачёта. На новом запуске исходный предмет снова доступен. Для короткой галочки достаточно общего индикатора предыдущего раздела; растворение — отдельный приём для карточки 5.

ПМ.13 · 23. УП.ПМ.13 · оформление и проверка

Экранные кнопки и удержание

1

Существующие кнопки

Canvas/GamePanel содержит Direction-0...Direction-3. У каждой Button, Image и DirectionalButton. На дочернем Label — стрелка. В DirectionalButton.Direction установлены Up (0,1), Down (0,-1), Left (-1,0), Right (1,0).

2

Восстановить отсутствующую кнопку

Новую кнопку создавать только если она заменяет отсутствующее направление. Сначала отключить или удалить прежнюю кнопку этого направления. На GamePanel: UI → Legacy → Button, имя ButtonUp; Rect Transform: Bottom Left, Pivot (0,0), Width/Height 72, Pos X/Pos Y 96. Add Component → DirectionalButton, Direction (0,1); Image.Raycast Target включён. Всего оставить четыре направления без наложения.

3

Почему OnClick недостаточно

OnClick срабатывает после отпускания. Для непрерывного движения DirectionalButton обрабатывает PointerDown, PointerUp и PointerExit. При выходе указателя или скрытии экрана направление сбрасывается.

4

Цветовые состояния

Button → Transition: Color Tint. ThemeRole применяет Normal, Highlighted и Pressed из палитры. Настраивать Surface, Accent и Success у VisualTheme. После запуска проверить наведение и удержание.

5

Проверка

Удерживать кнопку, отпустить, увести указатель. Движение прекращается. Поставить паузу при движении: накопленного направления после продолжения нет.

ПМ.13 · 24. УП.ПМ.13 · оформление и проверка

Звуковые события и источник сигнала

1

Единственный компонент

Выбрать GameManager → AudioFeedback и Audio Source. Play On Awake выключен, Spatial Blend = 0. Новый AudioFeedback добавлять не нужно: он уже связан с GameManager.Audio Feedback.

2

Импорт звука

Создать Assets/Audio/MyVariant, перетащить WAV/OGG. В Inspector: Load Type → Decompress On Load, Compression Format → PCM или Vorbis; нажать Apply, проверить прослушивание. Указать источник и лицензию в описании материалов.

3

Назначение

Перетащить файлы в Collect Clip, Warning Clip, Success Clip, Failure Clip. Пустые поля автоматически получают короткие синтезированные сигналы; это собственный программный материал, внешняя лицензия не нужна.

4

Причина и место вызова

Collect → PlayCollect; Warn → PlayHazard; Finish(true) → PlaySuccess; Finish(false) → PlayFailure. Не вызывать звук в Update и не вставлять второй вызов в обработчик: событие уже подключено.

5

Проверка

Проверить по одному событию каждого типа. Если тишина, проверить системную громкость, Mute Audio в Game, AudioListener на Main Camera и назначенный AudioSource. Наличие клипа не подтверждает слышимость.

ПМ.13 · 25. УП.ПМ.13 · оформление и проверка

Меню, пауза, результат и события кнопок

1

Четыре сохранённые панели

Canvas содержит MainMenuPanel, GamePanel, PausePanel и ResultPanel. Все ссылки ScreenMockupController заполнены до запуска. Не создавать ещё одну ResultPanel внутри Screens: HUD и контроллер должны использовать одну панель.

2

Проверка OnClick

Раскрыть PlayButton → Button → On Click: объект GameManager, метод GameManager.StartGame. PauseButton → PauseGame; ResumeButton → ResumeGame; MenuButton → GoToMenu; RestartButton → RestartGame. Чтобы восстановить событие, нажать «+», перетащить GameManager и выбрать метод в выпадающем списке.

3

Почему игра не зависает

GameManager.Start показывает меню и устанавливает остановку через ScreenMockupController. Нажатие «Играть» вызывает StartGame → ResetState → ResumeGame, которое устанавливает Time.timeScale = 1. При паузе IsPlaying выключается и направление сбрасывается.

4

Изменение компоновки

На MainMenuPanel задать название, цель и заметную кнопку запуска. PausePanel объясняет приостановку и содержит продолжение. ResultPanel показывает причину завершения и кнопку нового запуска. Сохранить общие поля шрифта, цвета и отступов.

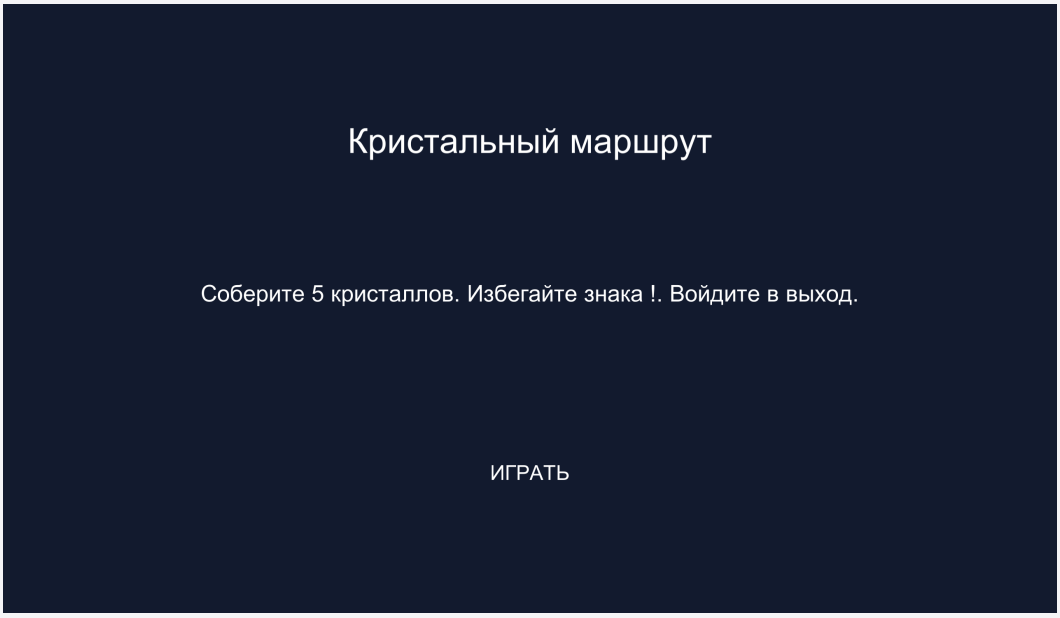
5

Проверка

Пройти меню → игра → пауза → продолжение → результат → заново. Прогресс на паузе не меняется. После результата управление остановлено. Остановить Play: Time.timeScale возвращается к 1.

6

Меню с целью и кнопкой запуска.



Меню с целью и кнопкой запуска.

7

Приостановка с продолжением и возвратом в меню.



Приостановка с продолжением и возвратом в меню.

8

Состояние победы и повторный запуск.



Состояние победы и повторный запуск.

9

Состояние поражения отличается текстом и причиной.



Состояние поражения отличается текстом и причиной.

ПМ.13 · 26. УП.ПМ.13 · оформление и проверка

Выбор и устройство варианта

1

Выбор номера

Остановить Play. В GameManager задать Variant Number от 1 до 30 по своей карточке. Номер 0 оставлен демонстрации. Поле Profiles ссылается на Assets/profiles.json; менять его путь не нужно.

2

Правило и данные

В profiles.json записи идут в порядке вариантов. title задаёт название, goal — условие, mode — готовое правило, target — цель, capacity — вместимость, resource — ограниченный ресурс. HUD читает состояние правила, поэтому цвет не подменяет механику.

3

Шесть готовых правил

mode 0 — принятие правильного типа; 1 — порядок №1–5; 2 — перенос по одному; 3 — груз ограниченной вместимости; 4 — три разных типа; 5 — разные переключатели и индикаторы доступа. Для ресурса действует дополнительная шкала 90…0, исчерпание завершает попытку.

4

Предел учебной модели

Числовые значения и условности модели заданы в карточке. Для комплекта каждому типу соответствует один номер; повторные предметы имеют тот же тип. В доступе индикаторы показывают разрешение выхода, физические стены уровня не перестраиваются. Это общая учебная основа для оформления разных состояний.

5

Что менять самостоятельно

Сохранить правило профиля; изменить title, goal и progressLabel в соответствующей записи Assets/profiles.json. Записи идут от варианта 1 до 30, номер 0 задан в SelectVariant. Сохранить JSON, вернуться в Unity и дождаться импорта. В именах и текстах оставить двойные кавычки JSON, запятые между свойствами; не удалять соседние записи. Соотнести номера с сущностями своего варианта в меню. Названия на самих предметах задаются отдельно после штатного присваивания SelectVariant.

6

Вариант 16: профиль переноса с ограниченной вместимостью.



Рисунок 8 — Оформленный вариант 16: доставлено 0/6, груз 0/3, три жизни. Это иллюстрация чтения вместимости, а не готовый индивидуальный результат.

ПМ.13 · 27. УП.ПМ.13 · оформление и проверка

Переход от демонстрации к своей области

1

Зафиксировать соответствие

По карточке составить соответствие «игровая сущность → объект сцены → поле HUD → событие → ожидаемый сигнал». Это позволяет понять, что именно оформлять и как проверить.

2

Выбрать дополнительное оформление

Реализовать обязательный визуальный сигнал области: форму, подпись, отметку прогресса и изменение после события. Для независимых отметок использовать строку состояния с номерами/галочками, для груза — число и пиктограмму, для порядка — выделение следующего номера.

3

Доработка HUD

В HUDView.ShowProgress можно изменить компоновку текста, сохраняя переданные title, progress и lives. Не вычислять игровой прогресс в HUD: источник данных — GameManager. Если добавили новое текстовое поле, создать объект Legacy Text, публичное поле Text и назначить ссылку в Inspector.

4

Пример осмысленной доработки

Для доставки добавить рядом с прогрессом подпись зоны приёмки; для порядка поставить номер следующей операции в заметную рамку; для доступа заменить GateIndicator на пиктограммы с теми же связями. Описать, что увидит пользователь до и после действия.

5

Повторная и регрессионная проверка

Проверить свой принятый и отклонённый случай, затем паузу, поражение, победу и сброс. Оформление нового показателя не должно скрывать жизни или перекрывать кнопки.

ПМ.13 · 28. Оформление игрового продукта

Постоянные ячейки, отметки и ресурс

1

Открыть данные представлению

В GameManager рядом с публичными Progress/Cargo/Lives добавить свойства ниже. Они только читают состояние правила. В новом Assets/Scripts/VariantPresentation.cs использовать полный листинг следующего раздела. На отдельный пустой объект Presentation назначить компонент; связать Manager, Player, Exit, предметы и индикаторы через Inspector.

2

Свойства для чтения

```
public float Remaining => remaining;
public bool HasFlag(int i) => i >= 0
    && i < flags.Length && flags[i];
public int NextIndex => progress;
```

3

Создать нужные индикаторы

Для груза создать дочернюю картинку на Player (Local Position 0,0.8,0) и ряд ячеек на HUD. Для комплекта/доступа — по одному GameObject с Image/SpriteRenderer на тип; рядом оставить постоянный контур и номер, а включаемой части дать галочку/заполнение. Для ресурса создать UI → Slider, Min 0, Max 90, Interactable выключен; рядом Legacy Text с названием ресурса. Число ячеек равно capacity, число отметок равно target. Ссылки неиспользуемых массивов оставить размером 0.

4

Соотнести особые сигналы

Загрузку над игроком или ячейки включать по Cargo; полный груз — отдельная рамка при Cargo == capacity. Для порядка подсветить только NextIndex, пройденные значки отметить по Progress. Для комплекта и доступа включать отдельную отметку по HasFlag(i), а оформление выхода — при Progress >= target. Низкий ресурс — Remaining <= 20; это порог оформления, правило по-прежнему заканчивается при 0. Не заменять живые данные постоянным текстом.

5

Оформить доступ

Индикаторы ворот/моста — изображения доступа, а не физические стены. В VariantPresentation массив Marks содержит заполненные линии/узоры, массив Empty содержит контуры/пунктир/полупрозрачные ворота; у одного типа общие символ и номер. Ready/Blocked на Exit показывают «Открыто»/замок; у варианта 30 промежуточные 0/2, 1/2 уже видны в HUD. У элементов UI отключить Raycast Target.

ПМ.13 · 29. Оформление игрового продукта

Подключение отображения состояния

1

VariantPresentation.cs

```
using UnityEngine;
using UnityEngine.UI;
public class VariantPresentation : MonoBehaviour
{
    public GameManager manager;
    public GameObject carried, fullCargo, ready, blocked, low;
    public GameObject[] cargoSlots = new GameObject[0];
    public GameObject[] marks = new GameObject[0];
    public GameObject[] empty = new GameObject[0];
    public GameObject[] next = new GameObject[0];
    public GameObject[] passed = new GameObject[0];
    public Slider resource;
    private void Update()
    {
        if (manager == null || manager.Current == null) return;
        var p = manager.Current;
        if (carried != null) carried.SetActive(manager.Cargo > 0);
        if (fullCargo != null) fullCargo.SetActive(
            p.capacity > 0 && manager.Cargo >= p.capacity);
        for (int i = 0; i < cargoSlots.Length; i++)
            cargoSlots[i].SetActive(i < manager.Cargo);
        for (int i = 0; i < marks.Length; i++)
            marks[i].SetActive(manager.HasFlag(i));
        for (int i = 0; i < empty.Length; i++)
            empty[i].SetActive(!manager.HasFlag(i));
        for (int i = 0; i < next.Length; i++)
            next[i].SetActive(i == manager.NextIndex);
        for (int i = 0; i < passed.Length; i++)
            passed[i].SetActive(i < manager.Progress);
        bool done = manager.Progress >= p.target;
        if (ready != null) ready.SetActive(done);
        if (blocked != null) blocked.SetActive(!done);
        if (resource != null) resource.value = manager.Remaining;
        if (low != null) low.SetActive(p.resource
            && manager.Remaining <= 20f);
    }
}
```

2

Сохранить ссылки

После компиляции назначить компонент на Presentation. Manager ← GameManager. Массивы заполнять соответствующими объектами в порядке номеров; Size = нужное число. В массив Next включать отдельные обводки, в Passed — галочки; не помещать туда корень Item, иначе скроется предмет с Collider. Для вариантов без такого показателя соответствующее поле оставить пустым. Сам Presentation оставить активным и поместить вне скрываемых панелей.

3

Проверить переходы

Для груза увидеть 0→1→capacity→0 после доставки; для отметок принять один тип и отклонить его повтор, затем принять другой — изменяются разные ячейки. Для ресурса сравнить Slider и текст HUD, поставить паузу — оба перестают убывать. После «Заново» проверить сброс всех индикаторов; если осталась галочка, проверить ссылку массива и не перепутан ли контур с заполнением.

ПМ.13 · 30. УП.ПМ.13 · оформление и проверка

Проверка проекта после сохранения

1

Рабочая последовательность

Сохранить Student. Очистить Console, запустить меню и игру. Повторить сценарии задания с ожидаемыми и фактическими результатами. При ошибке указать точный объект, компонент, поле или метод.

2

Повторное открытие

Остановить Play и снова открыть Student. Проверить палитру, ссылки HUD, OnClick и индексы предметов. Только затем повторить запуск. Этот тест отличает сохранённую настройку от временной правки в Play.

3

Сборка приложения

File → Build Profiles. Выбрать установленный профиль своего компьютера; в Scene List включить Student и исключить остальные сцены. Нажать Build, указать отдельную папку Build вне учебных исходников. Если модуль платформы не установлен, зафиксировать ограничение и не отмечать сборку выполненной.

4

Проверка сборки

Запустить полученное приложение и проверить меню, управление, принятую/отклонённую операцию, паузу и новый запуск. Работа в редакторе и запуск отдельного приложения — разные проверки; записать только фактически выполненное.

5

Заполнить и передать отчёт

Использовать отдельный «Отчет о выполнении.docx»: заполнить таблицы исходных условий и подстановок, разделы 2.1–2.6 и итоговую таблицу. Вставить собственные рисунки на подготовленные места; ожидаемые результаты записать до запуска, фактические — после. В разделе сдачи вписать ссылку на свой репозиторий. Версию Unity указать в таблице среды, дату проверки — рядом с фактическими результатами.

ПМ.13 · 31. УП.ПМ.13 · оформление и проверка

Диагностика характерных ошибок

1

Объект пропал после Play

В этой основе объекты должны быть сохранены в Student. Если исчезли все игровые объекты, проверить, не открыта ли старая пустая Main и не скопирован ли старый GameBootstrap. Открыть сохранённую сцену из исходного архива.

2

Текст не назначается в поле

Поле HUDView принимает UnityEngine.UI.Text. TMP_Text несовместим с ним. Выбрать существующий Legacy Text или создать именно UI → Legacy → Text. Не перетаскивать родительскую Panel.

3

Движение не возобновляется

Проверить PlayButton.OnClick → GameManager.StartGame и ResumeButton → ResumeGame. Изменение активности Panel само по себе не восстанавливает Time.timeScale.

4

Прогресс неверный

Проверить Variant Number, порядок GameManager.Items и Collectible.Index. Для последовательности принимается только следующий индекс, для комплекта повторный тип отклоняется, при переносе прогресс увеличивается в зоне доставки.

5

Пульсация портит коллайдер

Поле CollectibleVisual.Visual должно указывать на дочернее изображение. Корневой Item хранит Collider и не должен масштабироваться анимацией.

6

Цвет снова серый

В исходном Main нейтральная палитра. Изменить значения VisualTheme в Student и сохранить сцену вне Play. Не ограничиваться цветом Sprite Renderer.

ПМ.13 · 32. Оформление игрового продукта

Матрица проверок по правилам

1

Правильный тип (1–5)

До подбора — 0/5. Коснуться Item-1: 1/5 и предмет скрыт. Коснуться Item-6: 1/5 и предмет остаётся. Войти в Exit до 5/5: отказ. Принять 1–5 и войти в Exit: победа.

2

Порядок (6–10)

Новая попытка. Сначала Item-2: 0/5, он остаётся, требуется №1. Затем Item-1: 1/5, следующий №2. Последовательно 2–5, затем Exit: победа. Item-6 лишний; после 5/5 нужен Exit, а не шестой подбор.

3

Перенос (11–15)

Новая попытка. Коснуться Item-1: груз 1/1, доставлено 0/3. Item-2 до разгрузки отклонён, он остаётся. Войти в Exit: доставлено 1/3, груз 0. Повторить ещё две доставки разных предметов: победа. Пустая доставка не увеличивает прогресс.

4

Вместимость (16–20)

Взять capacity разных предметов: груз capacity/capacity, доставлено 0/6. Следующий предмет отклонён. Exit разгружает всё разом. Повторять, пока доставлено 6/6. Вместимости: 16→3, 17→2, 18→4, 19→5, 20→3. На последнем рейсе взять оставшиеся, полного груза не требуется.

5

Комплект (21–25)

Порядок типов предметов: 1,2,3,1,2,3. Принять Item-1: отмечен тип 1, прогресс 1/3. Коснуться Item-4: повтор типа 1 отклонён. Принять Item-2 и Item-3, войти в Exit: победа; три независимые отметки включены.

6

Доступ (26–30)

Для target=2 типы 1,2,1,2,1,2; для target=3 — 1,2,3,1,2,3. Принять Item-1, затем дубликат типа: счётчик прежний и показан отказ. Принять недостающие типы; индикаторы включаются независимо. Войти в Exit после target/target: победа. Физическая геометрия остаётся прежней.

7

Ресурс, урон и повтор

Для 17,18,20 отдельно начать новую попытку и не двигаться: ресурс убывает от 90 к 0, затем поражение; на паузе остаётся прежним. Во всех вариантах получить три контакта с Hazard с выходом из области и интервалом больше секунды: жизни 3→2→1→0, поражение. «Заново» возвращает весь исходный набор, «В меню» также сбрасывает попытку.

ПМ.13 · 33. Оформление игрового продукта

Подготовка репозитория и документов

1

Подготовить папку передачи

Остановить Play, сохранить Student, выполнить её повторное открытие. В корне рабочего проекта оставить Assets, Packages, ProjectSettings и .gitignore, исключающий Library/, Temp/, Logs/, obj/, UserSettings/, Build/. Сохранить .meta. В README указать Unity 6000.6.2f1, Assets/Scenes/Student.unity, номер, управление, правило, проверенные состояния и источники ресурсов.

2

Создать удалённый репозиторий

В GitHub после входа выбрать «+» → New repository, задать имя своего проекта, выбрать доступ для проверяющего. Не добавлять серверные README/.gitignore, если они уже подготовлены локально. В GitHub Desktop: File → Add local repository → Choose рабочую папку. Если Git ещё не создан, выбрать Create a repository here и проверить, что Local path указывает на корень проекта, без вложенного второго проекта. Проверить список изменений: служебных папок Unity быть не должно.

3

Опубликовать и проверить ссылку

В GitHub Desktop заполнить Summary, выполнить Commit to main, затем Publish repository (или Push origin, если remote уже назначен). Открыть View on GitHub. Для закрытого репозитория предоставить проверяющему доступ через Settings → Collaborators. В отчёт вставить URL страницы проекта. В отдельном браузерном сеансе убедиться, что доступны Student, скрипты и ресурсы; приватную ссылку проверяет пользователь с выданным доступом.

4

Заполнить чистую форму

Открыть «Отчет о выполнении.docx» из пакета, сохранить рабочую копию. Заполнить таблицы раздела 1, ответы и доказательства раздела 2, итоговую таблицу и поле репозитория раздела 3. В рамке удалить текст-подсказку и вставить Вставка → Рисунки → Это устройство; подпись под рамкой сохранить. Для широкого окна разрешена отдельная страница, без сжатия до нечитаемого текста. Ctrl+A → F9 обновляет номера и ссылки. Сохранить DOCX.

5

Проверить комплект

Заполнить индивидуальное задание и аттестационный лист по выданным формам, подготовить презентацию с собственными рисунками по её подписям; дополнительные формы пакета заполнить там, где это установлено карточкой. Сверить проект, отчёт и презентацию: один номер, одно название, одинаковые фактические результаты. Не включать чужой оформленный пример вместо Student. Передать документы и рабочую ссылку установленным способом сдачи.

ПМ.13 · 34. Справочник исходного кода

VisualTheme.cs

1

Файл и назначение

Файл Assets/Scripts/VisualTheme.cs. Полный исходный класс основы для сверки. Открыть существующий файл, найти нужный метод по имени; сохранять остальные методы и внешние скобки. Изменения из маршрута в этот исходный справочник не включены: он показывает первоначальную основу.

2

VisualTheme.cs

```
using UnityEngine;
using UnityEngine.UI;

public class VisualTheme : MonoBehaviour
{
    [Header("Палитра: роль цвета сохраняется на всех экранах")]
    public Color background = new Color(0.035f, 0.055f, 0.10f);
    public Color surface = new Color(0.07f, 0.10f, 0.18f);
    public Color player = new Color(0.19f, 0.88f, 1f);
    public Color target = new Color(1f, 0.78f, 0.16f);
    public Color danger = new Color(1f, 0.25f, 0.28f);
    public Color success = new Color(0.25f, 0.82f, 0.45f);
    public Color text = Color.white;
    public Color accent = new Color(0.16f, 0.55f, 0.75f);

    public void Apply()
    {
        Camera.main.backgroundColor = background;
        foreach (ThemeRole item in FindObjectsByType<ThemeRole>(FindObjectsInactive.
Include, FindObjectsSortMode.None))
        {
            Color color = item.role switch
            {
                ThemeRole.Role.Player => player,
                ThemeRole.Role.Target => target,
                ThemeRole.Role.Danger => danger,
                ThemeRole.Role.Success => success,
                ThemeRole.Role.Text => text,
                _ => surface
            };
            if (item.TryGetComponent(out SpriteRenderer sprite)) sprite.color =
color;
            if (item.TryGetComponent(out Graphic graphic)) graphic.color = color;
            if (item.TryGetComponent(out Button button))
            {
                if (item.TryGetComponent(out Graphic buttonGraphic))
buttonGraphic.color = Color.white;
                ColorBlock states = button.colors;
                states.normalColor = surface;
                states.highlightedColor = accent;
                states.pressedColor = success;
                button.colors = states;
            }
        }
    }
}
```

3

Назначение методов

Палитра хранится в полях Inspector; Apply выбирает роль и обновляет SpriteRenderer, Graphic и состояния Button. Значения Inspector сохраняются в сцене. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

4

Назначение методов — продолжение

Сначала задаётся фон камеры. Затем перебираются роли, выбирается цвет и обновляется графический компонент. У кнопки Graphic остаётся белым, а ColorBlock задаёт состояния: иначе цвета перемножатся и кнопка станет слишком тёмной. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

ПМ.13 · 35. Справочник исходного кода

ThemeRole.cs

1

Файл и назначение

Файл Assets/Scripts/ThemeRole.cs. Полный исходный класс основы для сверки. Открыть существующий файл, найти нужный метод по имени; сохранять остальные методы и внешние скобки. Изменения из маршрута в этот исходный справочник не включены: он показывает первоначальную основу.

2

ThemeRole.cs

```
using UnityEngine;
public class ThemeRole : MonoBehaviour
{
    public enum Role { Surface, Player, Target, Danger, Success, Text }
    public Role role;
}
```

3

Назначение методов

Компонент задаёт смысл цвета. Роль связывает оформление со значением объекта, не меняя игровые правила. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

ПМ.13 · 36. Справочник исходного кода

HUDView.cs

1

Файл и назначение

Файл Assets/Scripts/HUDView.cs. Полный исходный класс основы для сверки. Открыть существующий файл, найти нужный метод по имени; сохранять остальные методы и внешние скобки. Изменения из маршрута в этот исходный справочник не включены: он показывает первоначальную основу.

2

HUDView.cs

```
using UnityEngine;
using UnityEngine.UI;

public class HUDView : MonoBehaviour
{
    public Text titleText, progressText, livesText, hintText;
    public Text resultTitle, resultMessage, menuTitle, menuGoal;

    public void ShowProgress(string title, string progress, int lives)
    {
        titleText.text = title;
        progressText.text = progress;
        livesText.text = "Жизни: " + lives;
    }

    public void ShowHint(string message, bool warning)
    {
        hintText.text = (warning ? "! " : "✓ ") + message;
        hintText.color = warning ? new Color(1f, 0.5f, 0.4f) : Color.white;
    }

    public void SetGoal(string title, string goal)
    {
        menuTitle.text = title;
        menuGoal.text = goal;
    }

    public void ShowResult(bool won)
    {
        resultTitle.text = won ? "УСПЕХ ✓" : "ПОПРОБУЙТЕ ЕЩЁ !";
        resultMessage.text = won ? "Цель выполнена. Проверьте повторный запуск." : "
Ресурс исчерпан. Измените маршрут.";
    }
}
```

3

Назначение методов

Методы получают состояние извне. ShowProgress обновляет показатели, ShowHint различает принятие и отказ, ShowResult объясняет завершение. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

4

Назначение методов — продолжение

Три присваивания обновляют название, прогресс и жизни. Изменить формат одной подписи, затем проверить обновление при сборе и повреждении: здесь нельзя менять само число собранных предметов. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

5

Назначение методов — продолжение

Сначала составляется текст с признаком принятия/отказа; затем выбирается цвет предупреждения. Изменить только тело этого метода, сохранив параметры: GameManager уже передаёт message и warning. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

ПМ.13 · 37. Справочник исходного кода

CollectibleVisual.cs

1

Файл и назначение

Файл Assets/Scripts/CollectibleVisual.cs. Полный исходный класс основы для сверки. Открыть существующий файл, найти нужный метод по имени; сохранять остальные методы и внешние скобки. Изменения из маршрута в этот исходный справочник не включены: он показывает первоначальную основу.

2

CollectibleVisual.cs

```
using UnityEngine;

public class CollectibleVisual : MonoBehaviour
{
    [SerializeField] private Transform visual;
    [SerializeField] private float rotationSpeed = 70f;
    [SerializeField, Range(0f, 0.2f)] private float pulseAmount = 0.08f;
    [SerializeField, Min(0f)] private float pulseSpeed = 4f;
    private Vector3 initialScale;

    private void OnEnable()
    {
        if (visual != null) initialScale = visual.localScale;
    }

    private void Update()
    {
        if (visual == null) return;
        visual.Rotate(0f, 0f, rotationSpeed * Time.deltaTime);
        visual.localScale = initialScale * (1f + Mathf.Sin(Time.time * pulseSpeed)
* pulseAmount);
    }

    private void OnDisable()
    {
        if (visual == null) return;
        visual.localScale = initialScale;
        visual.localRotation = Quaternion.identity;
    }
}
```

3

Назначение методов

OnEnable сохраняет масштаб; Update изменяет только дочерний Visual; OnDisable восстанавливает исходный вид. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

4

Назначение методов — продолжение

Обновление использует deltaTime. Проверить, какой объект или состояние изменяется; в визуальной анимации это дочерний Visual, а в менеджере — ресурс и кнопка паузы. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

AudioFeedback.cs

1

Файл и назначение

Файл Assets/Scripts/AudioFeedback.cs. Полный исходный класс основы для сверки. Открыть существующий файл, найти нужный метод по имени; сохранять остальные методы и внешние скобки. Изменения из маршрута в этот исходный справочник не включены: он показывает первоначальную основу.

2

AudioFeedback.cs

```
using UnityEngine;

[RequireComponent(typeof(AudioSource))]
public class AudioFeedback : MonoBehaviour
{
    public AudioClip collectClip, warningClip, successClip, failureClip;
    private AudioSource source;

    private void Awake()
    {
        source = GetComponent<AudioSource>();
        source.playOnAwake = false;
        source.spatialBlend = 0f;
        if (collectClip == null) collectClip = Tone(880f);
        if (warningClip == null) warningClip = Tone(220f);
        if (successClip == null) successClip = Tone(1320f);
        if (failureClip == null) failureClip = Tone(110f);
    }

    public void PlayCollect() { source.PlayOneShot(collectClip); }
    public void PlayHazard() { source.PlayOneShot(warningClip); }
    public void PlaySuccess() { source.PlayOneShot(successClip); }
    public void PlayFailure() { source.PlayOneShot(failureClip); }

    private static AudioClip Tone(float frequency)
    {
        const int rate = 44100;
        float[] samples = new float[rate / 6];
        for (int i = 0; i < samples.Length; i++)
            samples[i] = Mathf.Sin(2f * Mathf.PI * frequency * i / rate) * 0.2f *
(1f - (float)i / samples.Length);
        AudioClip clip = AudioClip.Create("Учебный сигнал", samples.Length, 1,
rate, false);
        clip.SetData(samples, 0);
        return clip;
    }
}
```

3

Назначение методов

Awake создаёт сигналы для пустых полей. PlayOneShot вызывается только событием, Tone строит короткий затухающий звук. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

4

Назначение методов — продолжение

Массив хранит отсчёты синусоиды. Амплитуда уменьшается к концу, чтобы избежать резкого обрыва. Возвращённый AudioClip передаётся PlayOneShot. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

ПМ.13 · 39. Справочник исходного кода

ScreenMockupController.cs

1

Файл и назначение

Файл Assets/Scripts/ScreenMockupController.cs. Полный исходный класс основы для сверки. Открыть существующий файл, найти нужный метод по имени; сохранять остальные методы и внешние скобки. Изменения из маршрута в этот исходный справочник не включены: он показывает первоначальную основу.

2

ScreenMockupController.cs

```
using UnityEngine;

public class ScreenMockupController : MonoBehaviour
{
    public GameObject mainMenuPanel, gamePanel, pausePanel, resultPanel;

    public void ShowMainMenu() { SetOnly(mainMenuPanel); Time.timeScale = 0f; }
    public void ShowGame() { SetOnly(gamePanel); Time.timeScale = 1f; }
    public void ShowPause() { SetOnly(pausePanel); Time.timeScale = 0f; }
    public void ShowResult() { SetOnly(resultPanel); Time.timeScale = 0f; }

    private void SetOnly(GameObject selected)
    {
        foreach (GameObject panel in new[] { mainMenuPanel, gamePanel, pausePanel,
resultPanel })
            panel.SetActive(panel == selected);
    }

    private void OnDestroy() { Time.timeScale = 1f; }
}
```

3

Назначение методов

SetOnly оставляет один экран. Каждый публичный метод согласованно задаёт панель и масштаб времени. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

4

Назначение методов — продолжение

Все четыре панели перебираются, активной остаётся выбранная. Логика времени задаётся вызывающим методом. Проверка: одновременно виден один экран. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

ПМ.13 · 40. Справочник исходного кода

DirectionalButton.cs

1

Файл и назначение

Файл Assets/Scripts/DirectionalButton.cs. Полный исходный класс основы для сверки. Открыть существующий файл, найти нужный метод по имени; сохранять остальные методы и внешние скобки. Изменения из маршрута в этот исходный справочник не включены: он показывает первоначальную основу.

2

DirectionalButton.cs

```
using UnityEngine;
using UnityEngine.EventSystems;

public class DirectionalButton : MonoBehaviour, IPointerDownHandler,
IPointerUpHandler, IPointerExitHandler
{
    public Vector2 direction;
    public void OnPointerDown(PointerEventData data) {
GameManager.Instance.SetDirection(direction); }
    public void OnPointerUp(PointerEventData data) { Clear(); }
    public void OnPointerExit(PointerEventData data) { Clear(); }
    private void OnDisable() { Clear(); }
    private void Clear()
    {
        if (GameManager.Instance != null) GameManager.Instance.SetDirection(Vector2.
zero);
    }
}
```

3

Назначение методов

Удержание, отпускание, выход указателя и скрытие объекта определяют направление. Отпускание сбрасывает его. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

ПМ.13 · 41. Справочник исходного кода

CameraFit.cs

1

Файл и назначение

Файл Assets/Scripts/CameraFit.cs. Полный исходный класс основы для сверки. Открыть существующий файл, найти нужный метод по имени; сохранять остальные методы и внешние скобки. Изменения из маршрута в этот исходный справочник не включены: он показывает первоначальную основу.

2

CameraFit.cs

```
using UnityEngine;
[RequireComponent(typeof(Camera))]
public class CameraFit : MonoBehaviour
{
    private void LateUpdate()
    {
        var camera = GetComponent<Camera>();
        camera.orthographicSize = Mathf.Max(5.3f, 9f / camera.aspect);
    }
}
```

3

Назначение методов

Камера сохраняет поле целиком при изменении пропорций; Canvas независимо масштабирует экранный интерфейс. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

ПМ.13 · 42. Справочник исходного кода

PlayerController.cs

1

Файл и назначение

Файл Assets/Scripts/PlayerController.cs. Полный исходный класс основы для сверки. Открыть существующий файл, найти нужный метод по имени; сохранять остальные методы и внешние скобки. Изменения из маршрута в этот исходный справочник не включены: он показывает первоначальную основу.

2

PlayerController.cs

```
using UnityEngine;

[RequireComponent(typeof(Rigidbody2D), typeof(Collider2D))]
public class PlayerController : MonoBehaviour
{
    [SerializeField, Min(1f)] private float speed = 5f;
    private Rigidbody2D body;
    private Vector2 startPosition;

    private void Awake()
    {
        body = GetComponent<Rigidbody2D>();
        body.gravityScale = 0f;
        body.freezeRotation = true;
        startPosition = transform.position;
    }

    private void FixedUpdate()
    {
        if (GameManager.Instance == null || !GameManager.Instance.IsPlaying)
        {
            body.linearVelocity = Vector2.zero;
            return;
        }

        Vector2 keyboard = new Vector2(
            ReadAxis(KeyCode.A, KeyCode.LeftArrow, KeyCode.D, KeyCode.RightArrow),
            ReadAxis(KeyCode.S, KeyCode.DownArrow, KeyCode.W, KeyCode.UpArrow));
        Vector2 direction = Vector2.ClampMagnitude(
            keyboard + GameManager.Instance.MobileDirection, 1f);
        body.linearVelocity = direction * speed;
    }

    private static float ReadAxis(
        KeyCode negativeA, KeyCode negativeB,
        KeyCode positiveA, KeyCode positiveB)
    {
        float value = 0f;
        if (Input.GetKey(negativeA) || Input.GetKey(negativeB)) value -= 1f;
        if (Input.GetKey(positiveA) || Input.GetKey(positiveB)) value += 1f;
        return value;
    }

    public void ReturnToStart()
    {
        body.linearVelocity = Vector2.zero;
        body.position = startPosition;
    }
}
```

3

Назначение методов

FixedUpdate читает клавиатуру и экранное направление, ограничивает длину вектора; Rigidbody отвечает за движение и столкновения. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

4

Назначение методов — продолжение

Движение разрешено только в играющем состоянии. Вектор клавиатуры складывается с экранным направлением, ограничивается по длине и передаётся Rigidbody2D. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

ПМ.13 · 43. Справочник исходного кода

Collectible.cs

1

Файл и назначение

Файл Assets/Scripts/Collectible.cs. Полный исходный класс основы для сверки. Открыть существующий файл, найти нужный метод по имени; сохранять остальные методы и внешние скобки. Изменения из маршрута в этот исходный справочник не включены: он показывает первоначальную основу.

2

Collectible.cs

```
using UnityEngine;
[RequireComponent(typeof(Collider2D))]
public class Collectible : MonoBehaviour
{
    public int index;
    private void Awake() { GetComponent<Collider2D>().isTrigger = true; }
    private void OnTriggerEnter2D(Collider2D other)
    {
        if (other.GetComponent<PlayerController>() != null)
GameManager.Instance.Collect(this);
    }
}
```

3

Назначение методов

Триггер предмета передаёт конкретный объект менеджеру; индекс остаётся частью правила, а не оформления. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

ПМ.13 · 44. Справочник исходного кода

Hazard.cs

1

Файл и назначение

Файл Assets/Scripts/Hazard.cs. Полный исходный класс основы для сверки. Открыть существующий файл, найти нужный метод по имени; сохранять остальные методы и внешние скобки. Изменения из маршрута в этот исходный справочник не включены: он показывает первоначальную основу.

2

Hazard.cs

```
using UnityEngine;
[RequireComponent(typeof(Collider2D))]
public class Hazard : MonoBehaviour
{
    private void Awake() { GetComponent<Collider2D>().isTrigger = true; }
    private void OnTriggerEnter2D(Collider2D other)
    {
        if (other.GetComponent<PlayerController>() != null)
GameManager.Instance.HitHazard();
    }
}
```

3

Назначение методов

Касание игрока сообщает событие повреждения; число жизней и защита от повторного урона хранятся в менеджере. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

ПМ.13 · 45. Справочник исходного кода

ExitZone.cs

1

Файл и назначение

Файл Assets/Scripts/ExitZone.cs. Полный исходный класс основы для сверки. Открыть существующий файл, найти нужный метод по имени; сохранять остальные методы и внешние скобки. Изменения из маршрута в этот исходный справочник не включены: он показывает первоначальную основу.

2

ExitZone.cs

```
using UnityEngine;
[RequireComponent(typeof(Collider2D))]
public class ExitZone : MonoBehaviour
{
    private void Awake() { GetComponent<Collider2D>().isTrigger = true; }
    private void OnTriggerEnter2D(Collider2D other)
    {
        if (other.GetComponent<PlayerController>() != null)
GameManager.Instance.TryFinishLevel();
    }
}
```

3

Назначение методов

Вход игрока вызывает проверку цели либо доставку груза. Сам Collider не определяет победу. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

ПМ.13 · 46. Справочник исходного кода

GameManager.cs

1

Файл и назначение

Файл Assets/Scripts/GameManager.cs. Полный исходный класс основы для сверки. Открыть существующий файл, найти нужный метод по имени; сохранять остальные методы и внешние скобки. Изменения из маршрута в этот исходный справочник не включены: он показывает первоначальную основу.

2

GameManager.cs

```
using System;
using UnityEngine;

public class GameManager : MonoBehaviour
{
    public static GameManager Instance { get; private set; }
    public bool IsPlaying { get; private set; }
    public Vector2 MobileDirection { get; private set; }
    public PlayerController player;
    public HUDView hud;
    public ScreenMockupController screens;
    public VisualTheme theme;
    public AudioFeedback audioFeedback;
    public Collectible[] items;
    public TextAsset profiles;
    [Range(0, 30)] public int variantNumber;
    public SpriteRenderer[] gateIndicators;
    public int Progress => progress;
    public int Cargo => cargo;
    public int Lives => lives;
    public bool Won { get; private set; }
    public Profile Current => profile;

    [Serializable] public class Profile
    {
        public string title, goal, progressLabel;
        public int mode, target, capacity;
        public bool resource;
    }
    [Serializable] private class ProfileList { public Profile[] variants; }
    private Profile profile;
    private int progress, cargo, lives;
    private float immunity, remaining;
    private readonly bool[] flags = new bool[6];

    private void Awake() { Instance = this; }
    private void Start() { SelectVariant(variantNumber); screens.ShowMainMenu(); }
    private void Update()
    {
        if (Input.GetKeyDown(KeyCode.Escape))
        {
            if (IsPlaying) PauseGame(); else if (screens.pausePanel.activeSelf)
ResumeGame();
        }
        if (IsPlaying && profile.resource)
        {
            remaining -= Time.deltaTime;
            Refresh();
            if (remaining <= 0f) Finish(false);
        }
    }

    public void SelectVariant(int number)
    {
        variantNumber = number;
        profile = number == 0
```

2

GameManager.cs — продолжение

```
            ? new Profile { title = "Кристальный маршрут", goal = "Соберите 5
кристаллов. Избегайте знака !. Войдите в выход.", progressLabel = "Кристаллы", mode
= 0, target = 5, capacity = 0 }
            : JsonUtility.FromJson<ProfileList>(profiles.text).variants[number - 1];
        for (int i = 0; i < items.Length; i++)
        {
            bool warning = profile.mode == 0 && i >= profile.target;
            items[i].transform.Find("Visual").GetComponent<ThemeRole>().role =
warning ? ThemeRole.Role.Danger : ThemeRole.Role.Target;
            items[i].transform.Find("Number").GetComponent<TextMesh>().text =
warning ? "!" : ((profile.mode == 4 || profile.mode == 5) ? i % profile.target + 1 :
 i + 1).ToString();
        }
        theme.Apply();
        hud.SetGoal(profile.title, profile.goal);
        ResetState();
    }

    private void ResetState()
    {
        progress = cargo = 0; lives = 3; remaining = 90f; immunity = 0f;
        Won = false; IsPlaying = false; MobileDirection = Vector2.zero;
        Array.Clear(flags, 0, flags.Length);
        foreach (Collectible item in items) item.gameObject.SetActive(true);
        player.ReturnToStart();
        hud.ShowHint(profile.goal, false);
        Refresh();
    }

    public void StartGame() { ResetState(); ResumeGame(); }
    public void RestartGame() { StartGame(); }
    public void ResumeGame() { IsPlaying = true; screens.ShowGame(); }
    public void PauseGame() { IsPlaying = false; MobileDirection = Vector2.zero;
screens.ShowPause(); }
    public void GoToMenu() { ResetState(); screens.ShowMainMenu(); }
    public void SetDirection(Vector2 direction) { MobileDirection = IsPlaying ?
direction : Vector2.zero; }

    public void Collect(Collectible item)
    {
        if (!IsPlaying || !item.gameObject.activeSelf) return;
        int type = item.index % profile.target;
        bool wrong = profile.mode == 0 && item.index >= profile.target
            || profile.mode == 1 && item.index != progress
            || (profile.mode == 2 || profile.mode == 3) && cargo >= profile.capacity
            || (profile.mode == 4 || profile.mode == 5) && flags[type];
        if (wrong) { Warn("Действие не принято: проверьте порядок, тип или
вместимость."); return; }
        if (profile.mode == 2 || profile.mode == 3) cargo++;
        else { flags[type] = true; progress++; }
        item.gameObject.SetActive(false);
        audioFeedback.PlayCollect();
        hud.ShowHint("Принято. " + Status(), false);
        Refresh();
    }
```

2

GameManager.cs — продолжение

```
    public void TryFinishLevel()
    {
        if (!IsPlaying) return;
        if (profile.mode == 2 || profile.mode == 3)
        {
            if (cargo == 0) { Warn("Сначала возьмите груз."); return; }
            progress += cargo; cargo = 0;
            hud.ShowHint("Груз доставлен. " + Status(), false);
            Refresh();
            if (progress < profile.target) { audioFeedback.PlayCollect(); return; }
        }
        if (progress >= profile.target) Finish(true);
        else Warn("Цель ещё не достигнута. " + Status());
    }

    public void HitHazard()
    {
        if (!IsPlaying || Time.time < immunity) return;
        lives--; immunity = Time.time + 1f;
        player.ReturnToStart();
        if (lives == 0) Finish(false);
        else { Warn("Повреждение: потеряна жизнь."); Refresh(); }
    }

    private void Warn(string message) { audioFeedback.PlayHazard();
hud.ShowHint(message, true); }
    private string Status()
    {
        string text = profile.progressLabel + ": " + progress + "/" +
profile.target;
        if (profile.mode == 1) text += " · следующий №" + Mathf.Min(progress + 1,
profile.target);
        if (profile.mode == 2 || profile.mode == 3) text += " · груз " + cargo + "
/" + profile.capacity;
        if (profile.mode == 4 || profile.mode == 5)
        {
            text += " · ";
            for (int i = 0; i < profile.target; i++) text += (flags[i] ? "[✓" : "[ "
) + (i + 1) + "] ";
        }
        if (profile.resource) text += " · ресурс " + Mathf.CeilToInt(remaining);
        return text;
    }

    private void Refresh()
    {
        hud.ShowProgress(profile.title, Status(), lives);
        for (int i = 0; i < gateIndicators.Length; i++)
        {
            gateIndicators[i].gameObject.SetActive(profile.mode == 5 && i <
profile.target);
            gateIndicators[i].color = flags[i] ? theme.success : theme.danger;
        }
    }

    private void Finish(bool won)
```

2

GameManager.cs — продолжение

```
    {
        Won = won; IsPlaying = false; MobileDirection = Vector2.zero;
        if (won) audioFeedback.PlaySuccess(); else audioFeedback.PlayFailure();
        hud.ShowResult(won); screens.ShowResult();
    }
}
```

3

Назначение методов

Менеджер выбирает профиль, хранит прогресс, груз, жизни и флаги. Collect проверяет правило, Refresh обновляет HUD, Finish переключает экран. Все эти связи уже подготовлены в исходном проекте. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

4

Назначение методов — продолжение

Обновление использует deltaTime. Проверить, какой объект или состояние изменяется; в визуальной анимации это дочерний Visual, а в менеджере — ресурс и кнопка паузы. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

5

Назначение методов — продолжение

Сбрасываются прогресс, груз, жизни, ресурс и отметки; возвращаются предметы и игрок. После этого обновляется HUD. Проверка: новый запуск должен быть равен начальному состоянию. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

6

Назначение методов — продолжение

Сначала отсекаются остановленная игра и скрытый предмет. Затем проверяются тип, порядок, вместимость и повтор. Только принятое действие меняет состояние и скрывает объект; отклонённое вызывает Warn. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

7

Назначение методов — продолжение

Зона выхода для переноса сначала принимает груз и обнуляет его. Для прочих правил она проверяет цель. Успешная промежуточная доставка возвращается без сигнала ошибки; окончательная вызывает Finish. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

8

Назначение методов — продолжение

Выключаются игра и управление, выбирается один сигнал завершения, обновляется текст результата и включается его панель. Сохранить файл, дождаться компиляции и повторить связанный сценарий.

9

Границы правки

Для точной причины отказа заменяется только ветвь if (wrong) в Collect, для подписей добавляется строка в SelectVariant, для визуальных состояний — свойства чтения. Полный исходный класс не копировать поверх уже выполненных изменений.